
Chapter 03. Python Data Processing II

- 1. Pandas Data manipulation(조작)
 - 함수(Pandas U Function) 적용과 매핑_map, apply
 - 데이터 정렬
 - 기술 통계 요약
- 2. 파일 데이터 로딩과 저장 (엑셀, CSV 파일)
- 3. Concat, Merge (병합)

1. Pandas Data Manipulation

- pandas 객체에도 Numpy 유니버설 함수(배열 각 원소에 적용되는 메서드)를 적용해 데이터 처리 가능

1.1 numpy 유니버설 함수 리뷰

```
import numpy as np
import pandas as pd
```

```
arrn = np.random.randn(4,3)
print(arrn)
print('max value :', np.max(arrn))
print('min value :', np.min(arrn))
print('absolute value :')
print(np.abs(arrn))
```

(numpy 배열 절대값 계산)

```
[[ -0.89330098  0.7221263  -0.91336017]
 [ -1.10187112 -1.24287784 -0.77692892]
 [  1.34393072 -0.21582297 -0.50609732]
 [  1.8082162  -0.59200662  1.02334046]]
```

max value : 1.808216195949308

min value : -1.2428778428411353

```
absolute value :
[[ 0.89330098  0.7221263  0.91336017]
 [ 1.10187112  1.24287784  0.77692892]
 [ 1.34393072  0.21582297  0.50609732]
 [ 1.8082162   0.59200662  1.02334046]]
```

```
# axis 이용하여 연산 축 지정 가능
print(np.sum(arrn, axis = 1))
```

1.2 Series 및 DataFrame 에 Numpy U 함수 적용

```
import numpy as np
import pandas as pd

mydf = pd.DataFrame(np.random.randn(4,3), index=list('ABCD'),
                    columns = [ 'one','two','three'])

print( mydf )

print( np.abs(mydf) )           ( DataFrame 객체 절대값 계산 )

print( np.max( mydf['one'] ) )
```

('one' 컬럼 색인의 Series 객체
에 대한 최대값 계산)

	one	two	three
A	-2.211028	-0.598592	0.152952
B	0.835444	-0.486259	-0.681511
C	-0.725649	-0.751089	-0.775752
D	1.058090	0.127740	0.322206

	one	two	three
A	2.211028	0.598592	0.152952
B	0.835444	0.486259	0.681511
C	0.725649	0.751089	0.775752
D	1.058090	0.127740	0.322206

1.0580895674753898

1.3 pandas – map, apply, applymap 데이터 조작함수



메서드	통용대상(인수)	리턴 값	의미
map	Series(값 별)	Series	Series(1차원 배열의 각 값)에 대해 임의의 함수를 적용하고 1차원 배열로 값 리턴
apply	Dataframe(열 또는 행별)	Series,Dataframe	DataFrame 행축 또는 열축의 값에 대해 임의의 함수를 적용하고 1차원/2차원 배열로 새 객체 리턴

• lambda 함수 리뷰 및 map 메서드

```
# python Lambda exam
# lambda argument : expression
f = lambda x : x*3
print(f(3)) # lambda 함수에 3 전달
print( (lambda x : x**2)(5) ) # lambda 함수에 5 전달

# map 함수에 의해 형성된 sequence 데이터를 리스트 내포로 리
# 스트 형성
listdata = [ i for i in map( lambda x : x*3 , range(5) ) ]
print(listdata)
```

람다함수 X 에 3을 전달, X*3 을 리턴

람다함수 X 에 5을 전달, 5의 제곱승을 리턴

range(5) → 0~4 정수 데이터를 람다함수에
적용 → 0 3 6 9 12 데이터를 리스트 내포에
의해 리스트 데이터로 형성

python map() exam

map(function, sequence)

함수와 sequence형 데이터를 인자로 받아, 각 element마다 입력받은
함수에 적용하여 map iterator 객체로 반환

내부적으로 시퀀스 데이터를 순차적으로 호출함으로 for문을 사용하지
않아도 됨

```
listdata = [5,6,7,8,9]
```

```
lf = lambda x : x+3
```

```
print( map(lf, listdata) ) # list로 변환하지 않으면 map iterator 객체 반환
```

list로 변환 출력

```
print( list( map(lf, listdata) ) )
```

여러개 argument 사용

```
ld1 = [4, 6, 7, 9]
```

```
ld2 = [3, 1, 5, 2]
```

```
lf = lambda x,y : x-y
```

```
print( list( map(lf, ld1, ld2) ) )
```

<map object at
0x000001863A189438>

[8, 9, 10, 11, 12]

[1, 5, 2, 7]

1.4 map 메서드 활용

```
# Pandas Series type에 map 메서드 적용
# Series에 메서드 적용 시 인자값은 내부 값 사용
import numpy as np
import pandas as pd
s1 = pd.Series(np.arange(5), index = list('ABCDE'))
print(s1)
s2 = s1.map( lambda x : x**2 )
print( s2 )
```

(Series 각 요소에 lambda 함수 적용)

A	0
B	1
C	2
D	3
E	4

dtype: int32

A	0
B	1
C	4
D	9
E	16

dtype: int32

```
mydf = pd.DataFrame({ '철수' : [90, 85, 80, 75],
                      '영희' : [80, 90, 70, 65],
                      '길동' : [ 70, 80, 90, 100] },
                    index = ['국어','수학','영어','경제'])
print(mydf)
```

	철수	영희	길동
국어	90	80	70
수학	85	90	80
영어	80	70	90
경제	75	65	100

```
mydf['영희'] = mydf['영희'].map( lambda x : x+5 )
print(mydf)
```

('영희' 컬럼 데이터에 map 메서드
활용 가산점 5점 전체 적용)

	철수	영희	길동
국어	90	85	70
수학	85	95	80
영어	80	75	90
경제	75	70	100

• map 메서드 활용 (사전, Series)

```
mydf = pd.DataFrame({ '철수' : [90, 85, 80, 75],
                      '영희' : [80, 90, 70, 65],
                      '길동' : [ 70, 80, 90, 100] },
                      index = ['국어','수학','영어','경제'])
```

```
print(mydf)
```

Series 값을 사전의 키와 비교해서 매칭 되면 사전의 value로 교체
매칭 된 값이 없으면 NaN

```
dicdata = { 90:95, 85:90, 75: 80} # map 에 적용할 사전
print(mydf['철수'].map(dicdata))
```

```
sd1 = pd.Series([30,20,40,50],index=[90,65,80,75])
print(sd1)
print(mydf['철수'])
```

'철수'컬럼 Series 값과 sd1 시리즈 index 값과 매칭 되면 sd1 시리즈 value로 교체
매칭되지 않으면 NaN

```
print(mydf['철수'].map(sd1))
```

	철수	영희	길동
국어	90	80	70
수학	85	90	80
영어	80	70	90
경제	75	65	100

국어	95.0
수학	90.0
영어	NaN
경제	80.0

Name: 철수, dtype: float64

90	30
65	20
80	40
75	50

국어	90	국어	30.0
수학	85	수학	NaN
영어	80	영어	40.0
경제	75	경제	50.0

Series 데이터에 1:1 매칭 함수 적용이기 때문에 다음과 같은 코드는

```
print(mydf['철수'].map(lambda x : x.mean()))
```

 # 'int' object has no attribute 'mean' 오류

1.5 apply 메서드 활용

apply 메서드

: **DataFrame 행축 또는 열축의 값에 대해 임의의 함수를 적용**하고 1 또는 2차원 배열로 값 리턴

- axis = 0, axis = 1 연산 축 지정
- sum, mean과 같은 일반적인 통계 함수는 존재 함으로 apply 사용할 필요 없음

- DataFrame 객체에 apply(axis=0) 메서드 적용하면 모든 열을 분리 후 매핑 함수의 인자로 각 열(Series) 전달
- 매핑 함수에 따라 반환되는 객체 종류가 다름 (DataFrame Or Series)
- DataFrame 각 열(Series)을 매핑함수에 전달하고 Series 형태로 반환 되면 Series 형태가 하나의 DataFrame으로 통합
- DataFrame 각 열(Series)을 매핑함수에 전달하고 하나의 값을 반환하는 함수는 이들 값을 하나의 Series로 통합

- DataFrame 객체에 apply(axis=1) 적용하면 DataFrame 각 행을 매핑함수의 인자로 전달
- 행 인덱스가 매핑 결과로 반환 되는 Series의 인덱스
- Series 인덱스에 매칭되는 데이터 값은 각 행의 데이터를 함수에 적용한 리턴값

• DataFrame 객체 apply() 메서드 적용 예

```
import numpy as np
import pandas as pd

mydf = pd.DataFrame({ '철수' : [90, 85, 80, 75],
                      '영희' : [80, 90, 70, 65],
                      '길동' : [ 70, 80, 90, 100] },
                    index = ['국어','수학','영어','경제'])
```

```
print(mydf)
```

```
# apply 적용할 람다함수
```

```
lf = lambda x : x.max() - x.min()
```

```
print(mydf.apply(lf, axis = 0))
```

```
print(mydf.apply(lf, axis = 1)) # axis='columns' 과 동일
```

* 주의 : type이 맞지 않는 데이터가 있을 경우 오류

(로우 축 apply 메서드 적용 연산)

(컬럼 축 apply 메서드 적용 연산)

	철수	영희	길동
국어	90	80	70
수학	85	90	80
영어	80	70	90
경제	75	65	100

철수	15
영희	25
길동	30

dtype: int64

국어	20
수학	10
영어	20
경제	35

dtype: int64



1.6 Pandas Series, Dataframe 객체 정렬

• sort_index 메서드

: 로우나 컬럼 색인을 알파벳 순으로 정렬 후 새로운 객체 반환

```
import numpy as np
import pandas as pd
sd1 = pd.Series(np.arange(5,9), index = ['B','D','A','C'])
print(sd1)
```

B	5
D	6
A	7
C	8

dtype: int32

```
print(sd1.sort_index())
```

(색인 기준 알파벳 순으로 정렬)

A	7
B	5
C	8
D	6

dtype: int32

```
mydf = pd.DataFrame({ 'Lee' : [90, 85, 80, 75],
                      'Park' : [80, 90, 70, 65],
                      'Kim' : [ 70, 80, 90, 100] },
                    index = ['Kor','Eng','Sci','Math'])
```

```
print(mydf)
```

	Lee	Park	Kim
Kor	90	80	70
Eng	85	90	80
Sci	80	70	90
Math	75	65	100

```
print( mydf.sort_index() ) # default : axis = 0 로우 축 기준 정렬
```

```
# axis = 1 컬럼 축 기준 정렬, ascending = False : 내림차순
print( mydf.sort_index(axis = 1, ascending = False) )
```

	Lee	Park	Kim
Eng	85	90	80
Kor	90	80	70
Math	75	65	100
Sci	80	70	90

	Park	Lee	Kim
Kor	80	90	70
Eng	90	85	80
Sci	70	80	90
Math	65	75	100

• sort_values 메서드

: Series 객체를 값에 따라 정렬 후 새로운 객체 반환

```
sd2 = pd.Series( [5, 2, -7, 9] )
```

```
sd3 = sd2.sort_values()
```

(값을 기준으로 오름차순 정렬)

```
print(sd3)
```

drop=True : 인덱스 열을 컬럼열로 올리는데 아니라 버림

```
print( sd3.reset_index(drop=True) )
```

(index 초기화 하고 인덱스 열을 컬럼열로 올리는데 그냥 버림)

np.NaN은 기본적으로 맨 마지막에 위치

```
sd4 = pd.Series( [9, np.NaN, -6, 3, 8, np.NaN] )
```

```
sd5 = sd4.sort_values()
```

```
print(sd5)
```

```
print(mydf)
```

'Park' 컬럼 기준 데이터 정렬 , ascending = False 내림차순

```
sortdf = mydf.sort_values(by=['Park'], ascending = False)
```

```
print(sortdf)
```

by=['Park', ...] 표현은
첫컬럼기준 정렬 했을때 동일데이터가 있
을 경우 다음 컬럼 기준으로 정렬

('Park'컬럼 기준 내림차순 정렬)

2	-7
1	2
0	5
3	9

0	-7
1	2
2	5
3	9

2	-6.0
3	3.0
4	8.0
0	9.0
1	NaN
5	NaN
dtype: float64	

	Lee	Park	Kim
Eng	85	90	80
Kor	90	80	70
Sci	80	70	90
Math	75	65	100

1.7 기술 통계 요약 메서드

• 요약 통계 (sum)

```
import numpy as np
import pandas as pd
```

```
mydf = pd.DataFrame(np.arange(3,11).reshape(4,2),
                    index = list('ABCD'), columns = ['one','two'])
```

```
mydf.loc['A','two'] = np.NaN
```

```
mydf.loc['C'] = np.NaN
```

```
print(mydf)
```

(DataFrame 객체에 NaN 데이터 추가)

로우, 컬럼 축 데이터에 대한 합 계산

```
print(mydf.sum()) # 기본값 axis=0 로우 축 기준 합 Series 반환, 기본 NaN 값 제외
```

```
print(mydf.sum(axis=1)) # axis=1 컬럼 축 기준 합 Series 반환
```

skipna 옵션 : 누락된 값을 제외할 것인지 정하는 옵션, 기본값 True

```
print(mydf.sum(axis=1, skipna = False))
```

	one	two
A	3.0	NaN
B	5.0	6.0
C	NaN	NaN
D	9.0	10.0

	one	two
one	17.0	
two	16.0	

dtype: float64

(로우 축 기준 연산)

	A	B	C	D
one	3.0	11.0	0.0	19.0

dtype: float64

(컬럼 축 기준 연산)

	A	B	C	D
one	NaN	11.0	NaN	19.0

dtype: float64

• 기술 통계 요약 (mean, cumsum, describe)

로우, 컬럼 축 데이터 평균 계산

print(mydf.mean())

skipna = True : 누락된 값을 제외 처리

skipna = False : 누락된 값을 제외하지 말라

print(mydf.mean(axis=1, skipna = False))

(로우 축 기준 평균 연산)

(컬럼 축 기준 평균 연산,
NaN 제외 없이)

로우, 컬럼 축 데이터 누적합 계산(누산)

print(mydf)

print(mydf.cumsum()) # skipna=True 로 누락치 제외, 로우 축 기준 누산

print(mydf.cumsum(axis=1)) # 컬럼 축 기준 누산

여러 통계 데이터를 결과로 반환

print(mydf.describe())

one	5.666667
two	8.000000
dtype: float64	

A	NaN
B	5.5
C	NaN
D	9.5
dtype: float64	

	one	two
A	3.0	NaN
B	8.0	6.0
C	NaN	NaN
D	17.0	16.0

	one	two
A	3.0	NaN
B	5.0	11.0
C	NaN	NaN
D	9.0	19.0

	one	two
count	3.000000	2.000000
mean	5.666667	8.000000
std	3.055050	2.828427
min	3.000000	6.000000
25%	4.000000	7.000000
50%	5.000000	8.000000
75%	7.000000	9.000000
max	9.000000	10.000000

1.8 유일 값, 빈도 수 (1차원 Series에 담긴 값의 정보 추출)

```
sd1 = pd.Series( ['lee','kim','lee','park','hong','park','kim','kim'] )  
print(sd1)
```

```
unique_sd = sd1.unique()  
print(unique_sd)
```

(중복된 값을 제거하고 유일값만 담고 있는 Numpy 배열 반환)

```
value_cnt = sd1.value_counts()  
print(value_cnt)
```

(Series 데이터의 빈도수를 반환)

```
mydf = pd.DataFrame(np.random.rand(6,4),  
                    index=pd.date_range('20200301',periods=6),  
                    columns=list('ABCD'))
```

```
print(mydf)  
mydf['E'] = ['red','yellow','blue','red','pink','blue']  
print(mydf)
```

('E' 컬럼 데이터 추가)

* numpy unique() 메서드와 비교
unique_sd = np.unique(sd1, return_counts=True)
print(unique_sd)

['lee' 'kim' 'park' 'hong']

```
kim    3  
lee    2  
park   2  
hong   1  
dtype: int64
```

	A	B	C	D	E
2020-03-01	0.387095	0.188012	0.334731	0.880473	red
2020-03-02	0.556087	0.336175	0.354049	0.178756	yellow
2020-03-03	0.394954	0.490898	0.482499	0.077047	blue
2020-03-04	0.950719	0.238275	0.520273	0.120514	red
2020-03-05	0.243769	0.419469	0.610132	0.128027	pink
2020-03-06	0.433370	0.418962	0.134215	0.809794	blue

1.9 isin() 메서드

'E'컬럼 Series 에 'red','blue' 값이 존재하는지 나타내는 불리언 데이터 반환
`print(mydf['E'].isin(['red','blue']))`

불리언 색인 활용 True 인 항목만 추출
`print( mydf[ mydf['E'].isin(['red','blue']) ] )`

2020-03-01	True
2020-03-02	False
2020-03-03	True
2020-03-04	True
2020-03-05	False
2020-03-06	True

Freq: D, Name: E, dtype: bool

'E'컬럼 데이터 기준 항목 데이터 중
'red', 'blue' 가 존재하는 데이터 부분 추출

	A	B	C	D	E
2020-03-01	0.387095	0.188012	0.334731	0.880473	red
2020-03-03	0.394954	0.490898	0.482499	0.077047	blue
2020-03-04	0.950719	0.238275	0.520273	0.120514	red
2020-03-06	0.433370	0.418962	0.134215	0.809794	blue



isin 메서드 : 한 개 컬럼도 ['red'] 리스트 형태로 전달

2. 파일 데이터(csv,excel) 로딩 및 저장

- 표 형식 자료를 DataFrame 객체로 읽어오는 pandas 파일 파싱 함수

함수	설명
read_csv	파일,파일과 유사한 객체로부터 구분된 데이터를 로딩, 데이터 구분자는 쉼표(,)를 기본
read_table	파일,파일과 유사한 객체로부터 구분된 데이터를 로딩, 데이터 구분자는 탭('wt')를 기본
read_excel	엑셀 파일(xls, xlsx) 표 형식의 데이터 로딩
read_hdf	pandas에서 저장한 HDFS 파일에서 데이터 로딩
read_html	HTML 문서 내의 모든 테이블의 데이터를 로딩
read_json	JSON 문자열에서 데이터 로딩
read_pickle	파이썬 피클 포맷으로 저장된 객체를 로딩
read_sql	SQL 쿼리 결과를 pandas의 DataFrame형식으로 로딩

* 이외 더 많은 파일 파싱 함수 제공

2.1 read_csv 함수

: csv 파일을 읽어 pandas 기본 데이터 구조인 DataFrame 객체 생성

• read_csv 함수 기본 사용

```
import numpy as np
import pandas as pd
# 읽고자 하는 파일경로/파일명을 전달인자로 넘겨주면 됨
mydf = pd.read_csv("exam_1.csv") - 첫번째 로우를 기본 컬럼으로 지정
print(mydf) - header 옵션에 None 지정하면 컬럼 자동 생성

# header = None 옵션 : 첫번째 로우를 컬럼으로 사용하지 않고 자동 생성
mydf = pd.read_csv("exam_1.csv", header=None)
print(mydf)

# sep = ' ' 옵션 : 필드를 구분하기 위해 사용할 문자나 정규 표현식
mydf1 = pd.read_csv("exam_2.csv", sep=' ')
print(mydf1)
print(mydf1.columns) (필드를 콤마(,) 가 아닌 공백으로 구별)
```

exam_1.csv 파일 내용

```
a,b,c,d,message
1,2,3,4,hello
5,6,7,8,world
9,10,11,12,foo
```

```
   a  b  c  d message
0  1  2  3  4   hello
1  5  6  7  8   world
2  9 10 11 12    foo
```

```
   0  1  2  3  4
0 a  b  c  d message
1 1  2  3  4   hello
2 5  6  7  8   world
3 9 10 11 12    foo
```

```
   a  b  c  d message
0  1  2  3  4  python
1  5  6  7  8   pandas
2  9 10 11 12  library
```

exam_2.csv 파일 내용

```
a b c d message
1 2 3 4 python
5 6 7 8 pandas
9 10 11 12 library
```

• read_csv 함수 인자(옵션)

```
import numpy as np
import pandas as pd
# names : 컬럼 이름 지정 list
mydf = pd.read_csv('exam2.csv', names = ['age','height','weight','gender'])
print(mydf)
```

```
nameslist = ['age','height','weight','gender']
```

```
# index_col : 인덱스로 지정할 컬럼 선택 , age 컬럼으로 색인 지정
mydf = pd.read_csv('exam2.csv', names = nameslist, index_col='age')
print(mydf)
```

```
print(mydf_male.mean())
```

(남성 키,몸무게 평균 데이터)

```
height    176.50
weight     78.25
dtype: float64
```

exam2.csv 파일 내용

```
20,180,89,male
30,175,70,male
35,165,60,female
50,170,65,female
45,175,86,male
15,176,68,male
28,163,59,female
```

	age	height	weight	gender
0	20	180	89	male
1	30	175	70	male
2	35	165	60	female
3	50	170	65	female
4	45	175	86	male
5	15	176	68	male
6	28	163	59	female

gender : male 선택

	age	height	weight	gender
20	180	89	male	
30	175	70	male	
45	175	86	male	
15	176	68	male	

height weight gender

	age	height	weight	gender
20	180	89	male	
30	175	70	male	
35	165	60	female	
50	170	65	female	
45	175	86	male	
15	176	68	male	
28	163	59	female	

• read_csv 함수 인자(옵션)

```
import numpy as np
import pandas as pd
# nrows 인자 : 파일의 처음 몇 줄만 읽을 것인지 지정
mydf = pd.read_csv('exam2.csv', names = ['age','height','weight','gender'],
                  nrows=3)
print(mydf)
```

	age	height	weight	gender
0	20	180	89	male
1	30	175	70	male
2	35	165	60	female

(파일 전체 대신 처음 몇 줄만 읽어올지 지정)

```
# skiprows 인자 : 파일의 무시할 로우 번호가 담긴 리스트
mydf1 = pd.read_csv('exam2_skiprows.csv',
                  names = ['age','height','weight','gender'],
                  , skiprows=[0,3,4] )
print(mydf1)
```

exam2_skiprows.csv 파일 내용

```
# pandas data set
20,180,89,male
30,175,70,male
# skiprows option used
# read_csv
35,165,60,female
50,170,65,female
45,175,86,male
15,176,68,male
28,163,59,female
```

* 첫행부터~37000행 무시
하고 읽어올 경우
skiprows=list(range(0,37000))

	age	height	weight	gender
0	20	180	89	male
1	30	175	70	male
2	35	165	60	female
3	50	170	65	female
4	45	175	86	male
5	15	176	68	male
6	28	163	59	female

그 외 옵션

- thousands= ', ' : 데이터 천 단위 콤마(,) 제거

• read_csv 함수 인자(옵션)

* 대용량 csv 파일을 분할해서 읽을때 chunksize 파라미터 활용

```
import numpy as np
import pandas as pd
```

100 chunksize 씩 읽어 오기

```
def preprocessing(data):
```

```
    indexs = data.index.values
```

```
    print(len(indexs)) # 100
```

```
    print(indexs[:10], indexs[-10:])
```

```
    print(data.iloc[:10,:3])
```

```
    print('='*30)
```

[0 1 2 3 4 5 6 7 8 9] [90 91 92 93 94 95 96 97 98 99]

	PassengerId	Survived	Pclass
0	1	0	3
1	2	1	1
2	3	1	3
3	4	1	1
4	5	0	3
5	6	0	3
6	7	0	1
7	8	0	3
8	9	1	3
9	10	1	2

=====

~~~~~

```
for cnt, chunkdata in enumerate(pd.read_csv('titanic_passengers.csv', chunksize=100)):
```

```
    preprocessing(chunkdata)
```

```
    if cnt >= 10:
```

```
        break
```

100줄씩 \* cnt 변수 크기  
만큼 읽어옴

## • 한글 데이터 입/출력 시 인코딩 확인

```
import sys
```

```
# 파이썬 데이터 입출력 터미널 인코딩 확인
```

```
print(sys.stdin.encoding)
```

```
print(sys.stdout.encoding) # UTF-8
```

- 한글이 저장되는 인코딩 방식 ( Unicode, UTF-8, EUC-Kr, CP949 )
  - CP949(코드페이지949) : 마이크로소프트(MS)에서 도입한 문자 집합, EUC-Kr 확장,하위 호환
  - Unicode : 전세계 모든 언어에 겹치지 않는 코드를 할당(매핑)한 코드
  - UTF-8 : 유니코드 기반의 가변 길이 문자 인코딩 방식
- \* <https://redscreen.tistory.com/163> : 파이썬 한글 인코딩 탐구 사이트 참조

## # 한글 데이터 CSV 출력

# 윈도우 메모장, 엑셀에서 작성된 한글 포함 csv파일을 'CP949' encoding 방식을 지정하지 않을 경우 한글 깨짐, ( 하지만 메모장으로 확인된 encoding 방식이 UTF-8일 경우 깨지지 않음 )

```
data = pd.read_csv('seoul_keumchun_gas_20190916.csv', encoding='CP949')
```

```
print(data)
```

|   | 주유소명                | 대표자 ...      | 전화번호        | 데이터기준일자    |
|---|---------------------|--------------|-------------|------------|
| 0 | 태영유업(주)             | 강계영 ...      | 02-856-0283 | 2019-09-16 |
| 1 | KH에너지(주)직영 시흥대로 주유소 | 송진수 ...      | 02-892-5156 | 2019-09-16 |
| 2 | SK에너지(주)박미주유소       | 조경목 ...      | 02-802-6749 | 2019-09-16 |
| 3 | 지에스칼텍스(주)일신주유소      | 허세홍, 김형국 ... | 02-896-5145 | 2019-09-16 |
| 4 | SK네트웍스(주) 순환도로주유소   | 최신원, 박상규 ... | 02-859-5426 | 2019-09-16 |

메모장 다른이름 저장  
인코딩 확인

## 2.2 to\_csv 함수

: 데이터를 쉼표로 구분(csv)된 형식으로 파일 저장

### • csv 파일 저장

```
import numpy as np
import pandas as pd
perinfo = pd.DataFrame({'age':[20,30,35,55,40,15],
                        'height':[185,176,168,172,179,182],
                        'weight':[88,77,65,59,83,65]})
print(perinfo)
```

|   | age | height | weight |
|---|-----|--------|--------|
| 0 | 20  | 185    | 88     |
| 1 | 30  | 176    | 77     |
| 2 | 35  | 168    | 65     |
| 3 | 55  | 172    | 59     |
| 4 | 40  | 179    | 83     |
| 5 | 15  | 182    | 65     |

,age,height,weight  
0,20,185,88  
1,30,176,77  
2,35,168,65  
3,55,172,59  
4,40,179,83  
5,15,182,65

# 옵션을 명시하지 않으면 컬럼, 인덱스 저장

perinfo.to\_csv('person\_info\_data.csv')

( 쉼표로 구분된 파일로 내용 저장 )

# 인덱스, 컬럼 저장 하지 않을 경우

perinfo.to\_csv('person\_info\_data.csv', index = False,  
header = False)

20,185,88  
30,176,77  
35,168,65  
55,172,59  
40,179,83  
15,182,65

# 일부 컬럼만 저장할 경우

perinfo.to\_csv('person\_info\_data.csv', index=False,  
columns=['age','weight'])

age,weight  
20,88  
30,77  
35,65  
55,59  
40,83  
15,65

## • csv 파일 한글 내용 저장

```
import numpy as np
import pandas as pd
perinfo = pd.DataFrame({'age':[20,30,35,55,40,15],
                        'height':[185,176,168,172,179,182],
                        'weight':[88,77,65,59,83,65]})
```

( 한글 컬럼 데이터 추가 )

```
perinfo['성별'] = ['여성','남성','여성','남성','남성','여성']
print(perinfo)
```

# 한글 저장을 위한 encoding 방식 지정

```
perinfo.to_csv('person_info_data.csv',index=False,
encoding='CP949')
```

( 파일 저장 데이터 확인 )

# Series 데이터 저장

```
timedates = pd.date_range('20000101', periods = 7)
print(timedates)
ts = pd.Series(np.arange(7), index = timedates)
print(ts)
ts.to_csv('tseries.csv', header=False)
tsread = pd.read_csv('tseries.csv', header=None)
print(tsread)
```

( Series 데이터 파일 저장 확인 )

|   | age | height | weight | 성별 |
|---|-----|--------|--------|----|
| 0 | 20  | 185    | 88     | 여성 |
| 1 | 30  | 176    | 77     | 남성 |
| 2 | 35  | 168    | 65     | 여성 |
| 3 | 55  | 172    | 59     | 남성 |
| 4 | 40  | 179    | 83     | 남성 |
| 5 | 15  | 182    | 65     | 여성 |

| age,height,weight,성별 |
|----------------------|
| 20,185,88,여성         |
| 30,176,77,남성         |
| 35,168,65,여성         |
| 55,172,59,남성         |
| 40,179,83,남성         |
| 15,182,65,여성         |

|              |
|--------------|
| 2000-01-01,0 |
| 2000-01-02,1 |
| 2000-01-03,2 |
| 2000-01-04,3 |
| 2000-01-05,4 |
| 2000-01-06,5 |
| 2000-01-07,6 |

## 2.3 read\_table 함수

: 탭('Wt')를 기본 데이터 구분자로 읽어와 DataFrame 객체 생성

```
import numpy as np
import pandas as pd
```

```
mydf = pd.read_table("exam3.txt")
print(mydf)
```

# 필드가 탭이 아닌 여러개의 공백 문자로 구별되어 있을 경우 정규 표현식 `Ws+` 사용해 필드 구분

```
mydf1 = pd.read_table("exam4.txt", sep='Ws+')
```

```
print(mydf1)
print(mydf1.columns)
print(mydf1.index)
```

\* 정규식 종류는 별도 참조

|     | A         | B         | C         |
|-----|-----------|-----------|-----------|
| aaa | -0.264438 | -1.026059 | -0.619500 |
| bbb | -0.927272 | 0.302904  | -0.032399 |
| ccc | -0.279834 | -1.567605 | -0.198127 |
| ddd | -0.845721 | -0.232904 | 1.287455  |

tab 으로 구별된 exam3.txt 파일

|     | A         | B         | C         |
|-----|-----------|-----------|-----------|
| aaa | -0.264438 | -1.026059 | -0.619500 |
| bbb | -0.927272 | 0.302904  | -0.032399 |
| ccc | -0.279834 | -1.567605 | -0.198127 |
| ddd | -0.845721 | -0.232904 | 1.287455  |

|     | A         | B         | C         |
|-----|-----------|-----------|-----------|
| aaa | -0.264438 | -1.026059 | -0.619500 |
| bbb | -0.927272 | 0.302904  | -0.032399 |
| ccc | -0.279834 | -1.567605 | -0.198127 |
| ddd | -0.845721 | -0.232904 | 1.287455  |

여러공백으로 구별된 exam4.txt 파일

|     | A         | B         | C         |
|-----|-----------|-----------|-----------|
| aaa | -0.264438 | -1.026059 | -0.619500 |
| bbc | -0.927272 | 0.302904  | -0.032399 |
| ccc | -0.279834 | -1.567605 | -0.198127 |
| ddd | -0.845721 | -0.232904 | 1.287455  |



## 2.4 read\_excel 함수

: excel 데이터 읽어와 DataFrame 객체 생성 ( xlrd 모듈 필요 )

```
import numpy as np
import pandas as pd
```

```
# 서울 인구 수 읽어 오기
```

```
# sheet_name = '' : 읽어올 시트 이름
```

```
# header = 2 옵션 : 세번째 줄 부터 읽어 와라
```

```
# usecols = 'B, D, G, J, N' 옵션 : 엑셀 파일의 B, D, G, J, N 컬럼만 읽어 와라
```

```
# nrows : 파일의 몇 줄만 읽을것 인지 지정
```

```
popul_seoul_df = pd.read_excel("population_in_seoul.xls", sheet_name='서울시인구수',
                               header=2, usecols = 'B, D, G, J, N', nrows=6)
```

```
print(popul_seoul_df)
```

( 엑셀에 필요한 정보를 읽어 왔으나  
컬럼명이 보기 좋지 않아 변경 필요 )

|   | 자치구 | 계        | 계.1     | 계.2    | 65세이상고령자 |
|---|-----|----------|---------|--------|----------|
| 0 | 합계  | 10025927 | 9740398 | 285529 | 1468146  |
| 1 | 종로구 | 162860   | 151767  | 11093  | 27394    |
| 2 | 중구  | 136663   | 126409  | 10254  | 23025    |
| 3 | 용산구 | 244989   | 228830  | 16159  | 38531    |
| 4 | 성동구 | 311290   | 303158  | 8132   | 43662    |
| 5 | 광진구 | 368337   | 352692  | 15645  | 47347    |

# 컬럼 명 변경

```
popul_seoul_df.rename( columns = { popul_seoul_df.columns[0] : '지역구별',  
                                popul_seoul_df.columns[1] : '인구수',  
                                popul_seoul_df.columns[2] : '한국인',  
                                popul_seoul_df.columns[3] : '외국인',  
                                popul_seoul_df.columns[4] : '고령자'}, inplace = True )
```

print(popul\_seoul\_df)

( inplace = True 로 DataFrame 객체에  
변경 사항 반영 )

# 지역구별 컬럼을 인덱스로 재 설정

```
locindex_df = popul_seoul_df.set_index(['지역구별'])
```

print(locindex\_df)

|   | 지역구별 | 인구수      | 한국인     | 외국인    | 고령자     |
|---|------|----------|---------|--------|---------|
| 0 | 합계   | 10025927 | 9740398 | 285529 | 1468146 |
| 1 | 종로구  | 162860   | 151767  | 11093  | 27394   |
| 2 | 중구   | 136663   | 126409  | 10254  | 23025   |
| 3 | 용산구  | 244989   | 228830  | 16159  | 38531   |
| 4 | 성동구  | 311290   | 303158  | 8132   | 43662   |
| 5 | 광진구  | 368337   | 352692  | 15645  | 47347   |

# 합계 행 데이터 삭제

```
locindex_df.drop(['합계'], inplace = True)
```

print(locindex\_df)

|      | 인구수    | 한국인    | 외국인   | 고령자   |
|------|--------|--------|-------|-------|
| 지역구별 |        |        |       |       |
| 종로구  | 162860 | 151767 | 11093 | 27394 |
| 중구   | 136663 | 126409 | 10254 | 23025 |
| 용산구  | 244989 | 228830 | 16159 | 38531 |
| 성동구  | 311290 | 303158 | 8132  | 43662 |
| 광진구  | 368337 | 352692 | 15645 | 47347 |

## \* read\_excel ( ) warning 경우

```
warn("Workbook contains no default style, apply openpyxl's default")
```



해결안

### 1. 방법

```
import warnings
warnings.simplefilter("ignore")

excelfile = pd.read_excel(file, engine="openpyxl")
```

### 2. 방법

Excel 파일의 시트 이름을 "시트1"에서 "Sheet1"로 업데이트

## \* 대용량 excel 파일 로드 경우

- chunksize 파라미터가 없으므로 skiprows 와 nrow 파라미터 활용해 데이터 분할 로드 해야함

```
import numpy as np
import pandas as pd

skiprow = 0
nrow = 100
youtubedata_header = pd.read_excel('youtube_rank_1000.xlsx', nrow=0)
print(youtubedata_header.columns) # header 부분만 추출
youtubedata = pd.read_excel('youtube_rank_1000.xlsx', nrow=nrow)
print(youtubedata) # 초기 nrow 크기만큼 로드
print('=' * 80)

for x in range(1,5): # 테스트로 5회 반복
    skiprow = skiprow + nrow
    youtubedata = pd.read_excel('youtube_rank_1000.xlsx', skiprows=skiprow, nrow=nrow)
    print(youtubedata)
    print('='*80)
```

## 2.5 to\_excel 함수

: DataFrame 객체를 excel 파일로 저장 ( openpyxl 모듈 필요 )

# 시트네임 미지정시 디폴트 sheet1

# DataFrame 내용을 지정한 엑셀 파일로 저장

# index = False : 인덱스 저장 하지 않음

locindex\_df.to\_excel('seoul\_local\_population1.xlsx', sheet\_name = '서울지역별인구수')

( 앞 PPT에서 정리한 DataFrame 객체 내용을  
엑셀 파일로 저장,  
sheet\_name은 지정한 이름으로 저장 )

|    | A    | B      | C      | D     | E     |
|----|------|--------|--------|-------|-------|
| 1  | 지역구별 | 인구수    | 한국인    | 외국인   | 고령자   |
| 2  | 종로구  | 162860 | 151767 | 11093 | 27394 |
| 3  | 중구   | 136663 | 126409 | 10254 | 23025 |
| 4  | 용산구  | 244989 | 228830 | 16159 | 38531 |
| 5  | 성동구  | 311290 | 303158 | 8132  | 43662 |
| 6  | 광진구  | 368337 | 352692 | 15645 | 47347 |
| 7  | 동대문구 | 363779 | 346551 | 17228 | 58764 |
| 8  | 중랑구  | 403776 | 398812 | 4964  | 64449 |
| 9  | 성북구  | 454114 | 441590 | 12524 | 70204 |
| 10 | 강북구  | 317875 | 314090 | 3785  | 59808 |
| 11 | 도봉구  | 337594 | 335280 | 2314  | 58070 |
| 12 | 노원구  | 539829 | 535282 | 4547  | 79968 |
| 13 | 은평구  | 486159 | 481663 | 4496  | 79957 |
| 14 | 서대문구 | 324089 | 310069 | 14020 | 51708 |
| 15 | 마포구  | 385605 | 373629 | 11976 | 51961 |
| 16 | 양천구  | 463931 | 459849 | 4082  | 60487 |
| 17 | 강서구  | 600240 | 593708 | 6532  | 82937 |
| 18 | 구로구  | 439850 | 406748 | 33102 | 64952 |
| 19 | 금천구  | 252868 | 233371 | 19497 | 37191 |
| 20 | 영등포구 | 402478 | 368402 | 34076 | 57350 |
| 21 | 동작구  | 410487 | 397618 | 12869 | 61607 |
| 22 | 관악구  | 519953 | 502117 | 17836 | 74518 |
| 23 | 서초구  | 435258 | 431027 | 4231  | 56455 |
| 24 | 강남구  | 549043 | 544028 | 5015  | 70029 |
| 25 | 송파구  | 684338 | 677489 | 6849  | 86062 |
| 26 | 강동구  | 430522 | 426219 | 4303  | 61710 |

### 3. 데이터 조인(병합), 변형

#### 3.1 pandas merge

- 하나 이상의 키를 기준으로 DataFrame의 로우를 병합

```
import numpy as np
import pandas as pd

mydf1 = pd.DataFrame( {'customerid':[10,20,30,40,50,60,70],
                       'name':['Kim','Lee','Hong','Park','Kim','Han','Lee']})
```

```
print(mydf1)
```

```
mydf2 = pd.DataFrame( {'customerid':[20,20,40,90,50],
                       'deposit':[6000,8500,7200,4500,9200]})
print(mydf2)
```

# merge() : 기본 중복된 컬럼 이름을 키로 사용 , on = 'customerid'  
로 명시가 가독성 향상

# 디폴트 : inner join : 양쪽 테이블 모두에 존재하는 키 조합 사용

```
mergedf = pd.merge(mydf1,mydf2)
print(mergedf)
```

|   | customerid | name |
|---|------------|------|
| 0 | 10         | Kim  |
| 1 | 20         | Lee  |
| 2 | 30         | Hong |
| 3 | 40         | Park |
| 4 | 50         | Kim  |
| 5 | 60         | Han  |
| 6 | 70         | Lee  |

|   | customerid | deposit |
|---|------------|---------|
| 0 | 20         | 6000    |
| 1 | 20         | 8500    |
| 2 | 40         | 7200    |
| 3 | 90         | 4500    |
| 4 | 50         | 9200    |

|   | customerid | name | deposit |
|---|------------|------|---------|
| 0 | 20         | Lee  | 6000    |
| 1 | 20         | Lee  | 8500    |
| 2 | 40         | Park | 7200    |
| 3 | 50         | Kim  | 9200    |

( customerid 20은 오른쪽 DataFrame에 2개의 로우가 있음으로 데카르트곱 반환 , (Lee, 6000), (Lee, 8500) )

## • pandas merge 함수 인자 활용

```
mydf1 = pd.DataFrame( {'customerid':[10,20,30,40,50,60,70],  
                        'name':['Kim','Lee','Hong','Park','Kim','Han','Lee']})
```

```
print(mydf1)
```

```
mydf2 = pd.DataFrame( {'customeridtmp':[20,20,40,90,50],  
                        'deposit':[6000,8500,7200,4500,9200]})
```

```
print(mydf2)
```

#중복된 컬럼이 없을 경우 오류, left\_on, right\_on 옵션으로 지정해야 함

```
mergedf =
```

```
pd.merge(mydf1,mydf2,left_on='customerid',right_on='customeridtmp')
```

```
print(mergedf)
```

|   | customerid | name |
|---|------------|------|
| 0 | 10         | Kim  |
| 1 | 20         | Lee  |
| 2 | 30         | Hong |
| 3 | 40         | Park |
| 4 | 50         | Kim  |
| 5 | 60         | Han  |
| 6 | 70         | Lee  |

|   | customeridtmp | deposit |
|---|---------------|---------|
| 0 | 20            | 6000    |
| 1 | 20            | 8500    |
| 2 | 40            | 7200    |
| 3 | 90            | 4500    |
| 4 | 50            | 9200    |

|   | customerid | name | customeridtmp | deposit |
|---|------------|------|---------------|---------|
| 0 | 20         | Lee  | 20            | 6000    |
| 1 | 20         | Lee  | 20            | 8500    |
| 2 | 40         | Park | 40            | 7200    |
| 3 | 50         | Kim  | 50            | 9200    |

# how = 'left' 옵션 : 왼쪽 테이블에 존재하는 모든 키 조합을 사용

```
mergedf = pd.merge(mydf1,mydf2, how='left')  
print(mergedf)
```

# how = 'right' 옵션 : 오른쪽 테이블에 존재하는 모든 키 조합을 사용

```
mergedf = pd.merge(mydf1,mydf2, how='right')  
print(mergedf)
```

# how = 'outer' 옵션 : 양쪽 테이블에 존재하는 모든 키 조합을 사용

# 두 테이블 데이터의 중복된 컬럼이 있어야함( costomerid )

```
mergedf = pd.merge(mydf1,mydf2, how='outer')  
print(mergedf)
```

- outer(외부조인)은 합집합 결과 반환
- inner(내부조인)은 교집합 결과 반환

|   | customerid | name | deposit |
|---|------------|------|---------|
| 0 | 10         | Kim  | NaN     |
| 1 | 20         | Lee  | 6000.0  |
| 2 | 20         | Lee  | 8500.0  |
| 3 | 30         | Hong | NaN     |
| 4 | 40         | Park | 7200.0  |
| 5 | 50         | Kim  | 9200.0  |
| 6 | 60         | Han  | NaN     |
| 7 | 70         | Lee  | NaN     |

|   | customerid | name | deposit |
|---|------------|------|---------|
| 0 | 20         | Lee  | 6000    |
| 1 | 20         | Lee  | 8500    |
| 2 | 40         | Park | 7200    |
| 3 | 50         | Kim  | 9200    |
| 4 | 90         | NaN  | 4500    |

|   | customerid | name | deposit |
|---|------------|------|---------|
| 0 | 10         | Kim  | NaN     |
| 1 | 20         | Lee  | 6000.0  |
| 2 | 20         | Lee  | 8500.0  |
| 3 | 30         | Hong | NaN     |
| 4 | 40         | Park | 7200.0  |
| 5 | 50         | Kim  | 9200.0  |
| 6 | 60         | Han  | NaN     |
| 7 | 70         | Lee  | NaN     |
| 8 | 90         | NaN  | 4500.0  |



## 3.2 pandas concat 함수

- 데이터 합치기 , numpy concatenate와 유사



### • Series 간 concat

```
import numpy as np
import pandas as pd
```

```
s1 = pd.Series([0,1], index = ['a', 'b'])
```

```
print(s1)
```

```
s2 = pd.Series([2,3,4], index = ['c','d','e'])
```

```
print(s2)
```

```
s3 = pd.Series([5,6], index = ['f','g'])
```

```
print(s3)
```

# 열(컬럼)축 기준으로 붙이기, axis=1 인 경우 DataFrame 생성

```
s_concat = pd.concat([s1,s2,s3], axis = 1)
```

```
print(s_concat)
```

# 로우축 기준으로 붙이기, 디폴트 axis = 0, 새로운 Series 객체 생성

```
s_concat = pd.concat([s1,s2,s3], axis = 0 )
```

```
print(s_concat)
```

```
a  0
b  1
dtype: int64
```

```
c  2
d  3
e  4
dtype: int64
```

```
f  5
g  6
dtype: int64
```

|   | 0   | 1   | 2   |
|---|-----|-----|-----|
| a | 0.0 | NaN | NaN |
| b | 1.0 | NaN | NaN |
| c | NaN | 2.0 | NaN |
| d | NaN | 3.0 | NaN |
| e | NaN | 4.0 | NaN |
| f | NaN | NaN | 5.0 |
| g | NaN | NaN | 6.0 |

```
a  0
b  1
c  2
d  3
e  4
f  5
g  6
dtype: int64
```

## • Series 간 concat , join 옵션

```
s1 = pd.Series([8,3], index = ['a', 'b'])
```

```
print(s1)
```

```
s3 = pd.Series([5,6], index = ['f','g'])
```

```
print(s3)
```

```
s4 = pd.concat([s1,s3])
```

```
print(s4)
```

```
s5 = pd.concat([s1, s4], axis = 1)
```

```
print(s5)
```

```
# join = 'inner' : 내부조인(교집합)
```

```
# join = 'outer' : 외부조인(합집합) , 기본값은 'outer'
```

```
s5 = pd.concat([s1, s4], axis = 1, join = 'inner')
```

```
print(s5)
```

```
a  8  
b  3  
dtype: int64
```

```
f  5  
g  6  
dtype: int64
```

```
a  8  
b  3  
f  5  
g  6  
dtype: int64
```

```
0 1  
a 8.0 8  
b 3.0 3  
f NaN 5  
g NaN 6
```

```
0 1  
a 8 8  
b 3 3
```

(열축 기준, 기본 outer 결합)



## • Series 간 concat , keys 옵션

# keys옵션을 활용 Series 간 구별을 위해 계층적 색인 생성

```
pd_concat = pd.concat([s1,s2,s3], keys = ['one','two','three'])  
print(pd_concat)
```

|       |   |   |
|-------|---|---|
| one   | a | 8 |
|       | b | 3 |
| two   | c | 2 |
|       | d | 3 |
|       | e | 4 |
| three | f | 5 |
|       | g | 6 |

dtype: int64

# Series를 axis = 1로 병합할 경우 keys는 DataFrame의 컬럼 제목이 됨

```
pd_concat = pd.concat([s1,s2,s3],axis=1, keys = ['one','two','three'])  
print(pd_concat)
```

|   | one | two | three |
|---|-----|-----|-------|
| a | 8.0 | NaN | NaN   |
| b | 3.0 | NaN | NaN   |
| c | NaN | 2.0 | NaN   |
| d | NaN | 3.0 | NaN   |
| e | NaN | 4.0 | NaN   |
| f | NaN | NaN | 5.0   |
| g | NaN | NaN | 6.0   |

# reset\_index()활용 인덱스 초기화

# drop=True 옵션 : 기존인덱스를 컬럼열로 올리지 말고 삭제

```
reindexdf = pd_concat.reset_index(drop=True)  
print(reindexdf)
```

|   | one | two | three |
|---|-----|-----|-------|
| 0 | 8.0 | NaN | NaN   |
| 1 | 3.0 | NaN | NaN   |
| 2 | NaN | 2.0 | NaN   |
| 3 | NaN | 3.0 | NaN   |
| 4 | NaN | 4.0 | NaN   |
| 5 | NaN | NaN | 5.0   |
| 6 | NaN | NaN | 6.0   |

## • DataFrame 간 축 기준 concat , keys 옵션

```
import numpy as np
import pandas as pd
```

```
df1 = pd.DataFrame(np.arange(6).reshape(3,2), index=['a','b','c'],
                    columns = ['Data1','Data2'])
print(df1)
```

|   | Data1 | Data2 |
|---|-------|-------|
| a | 0     | 1     |
| b | 2     | 3     |
| c | 4     | 5     |

```
df2 = pd.DataFrame(np.arange(5,9).reshape(2,2), index = ['a','c'],
                    columns = ['Data3','Data4'])
print(df2)
```

|   | Data3 | Data4 |
|---|-------|-------|
| a | 5     | 6     |
| c | 7     | 8     |

# 로우 축 병합

```
df_concat = pd.concat([df1,df2], axis = 0)
print(df_concat)
```

|   | Data1 | Data2 | Data3 | Data4 |
|---|-------|-------|-------|-------|
| a | 0.0   | 1.0   | NaN   | NaN   |
| b | 2.0   | 3.0   | NaN   | NaN   |
| c | 4.0   | 5.0   | NaN   | NaN   |
| a | NaN   | NaN   | 5.0   | 6.0   |
| c | NaN   | NaN   | 7.0   | 8.0   |

# 컬럼 축 병합

```
df_concat = pd.concat([df1,df2], axis = 1)
print(df_concat)
```

|   | Data1 | Data2 | Data3 | Data4 |
|---|-------|-------|-------|-------|
| a | 0     | 1     | 5.0   | 6.0   |
| b | 2     | 3     | NaN   | NaN   |
| c | 4     | 5     | 7.0   | 8.0   |

# keys 옵션을 활용 계층 색인

```
df_concat = pd.concat([df1,df2], axis = 1, keys = ['level1','level2'])
print(df_concat)
```

|   | level1 |       | level2 |       |
|---|--------|-------|--------|-------|
|   | Data1  | Data2 | Data3  | Data4 |
| a | 0      | 1     | 5.0    | 6.0   |
| b | 2      | 3     | NaN    | NaN   |
| c | 4      | 5     | 7.0    | 8.0   |

## • DataFrame 간 concat 병합 예제

```
import numpy as np
import pandas as pd

gas_station = pd.DataFrame({'주유소상호':['동일','아리랑','장위','세창'],
                           '주소':['보문로','아리랑로','화랑로','동소문로'],
                           '세창장유무':['유','유','유','무']})

print(gas_station)

gas_station_position = pd.DataFrame({'위도':[37.58295759,37.60053504,37.60346513,37.60880691],
                                     '경도':[127.0203742,127.0142141,127.0434457,127.0294733]})

print(gas_station_position)
gas_station_info = pd.concat([gas_station,gas_station_position], axis=1)
print(gas_station_info)
```

|   | 주유소상호 | 주소   | 세창장유무 |
|---|-------|------|-------|
| 0 | 동일    | 보문로  | 유     |
| 1 | 아리랑   | 아리랑로 | 유     |
| 2 | 장위    | 화랑로  | 유     |
| 3 | 세창    | 동소문로 | 무     |

|   | 위도        | 경도         |
|---|-----------|------------|
| 0 | 37.582958 | 127.020374 |
| 1 | 37.600535 | 127.014214 |
| 2 | 37.603465 | 127.043446 |
| 3 | 37.608807 | 127.029473 |

axis = 1 ( 컬럼열 기준 병합 )



|   | 주유소상호 | 주소   | 세창장유무 | 위도        | 경도         |
|---|-------|------|-------|-----------|------------|
| 0 | 동일    | 보문로  | 유     | 37.582958 | 127.020374 |
| 1 | 아리랑   | 아리랑로 | 유     | 37.600535 | 127.014214 |
| 2 | 장위    | 화랑로  | 유     | 37.603465 | 127.043446 |
| 3 | 세창    | 동소문로 | 무     | 37.608807 | 127.029473 |

## • DataFrame 간 concat , ignore\_index 옵션

```
import numpy as np
import pandas as pd
```

```
mydf1 = pd.DataFrame(np.random.randn(3,4), columns = ['a','b','c','d'])
print(mydf1)
```

|   | a         | b         | c         | d         |
|---|-----------|-----------|-----------|-----------|
| 0 | -0.430458 | 0.889487  | 0.693491  | -1.088471 |
| 1 | -2.081982 | -0.159880 | 0.142078  | 1.707725  |
| 2 | 0.532631  | -0.515385 | -0.022984 | 0.253035  |

```
mydf2 = pd.DataFrame(np.random.randn(2,3), columns = ['b','d','a'])
print(mydf2)
```

|   | b         | d        | a         |
|---|-----------|----------|-----------|
| 0 | -0.123566 | 0.580798 | -1.557697 |
| 1 | -1.422044 | 0.350228 | 0.913058  |

```
# ignore_index : 이어붙은 축색인 무시 하고 새로운 색인 생성
mydf_concat = pd.concat([mydf1, mydf2], ignore_index = True )
print(mydf_concat)
```

|   | a         | b         | c         | d         |
|---|-----------|-----------|-----------|-----------|
| 0 | -0.430458 | 0.889487  | 0.693491  | -1.088471 |
| 1 | -2.081982 | -0.159880 | 0.142078  | 1.707725  |
| 2 | 0.532631  | -0.515385 | -0.022984 | 0.253035  |
| 3 | -1.557697 | -0.123566 | NaN       | 0.580798  |
| 4 | 0.913058  | -1.422044 | NaN       | 0.350228  |

```
# axis=1: 컬럼축 병합, ignore_index=True : 새로운 색인 인덱스
mydf_concat = pd.concat([mydf1, mydf2], axis=1, ignore_index=True )
print(mydf_concat)
```

|   | 0         | 1        | 2         | 3        | 4         | 5        | 6         |
|---|-----------|----------|-----------|----------|-----------|----------|-----------|
| 0 | -0.415664 | 1.838023 | -0.698019 | 0.080199 | -0.136617 | 0.206121 | -0.501155 |
| 1 | -1.560030 | 0.875148 | -1.701801 | 0.430441 | 0.321412  | 2.025529 | -1.426448 |
| 2 | 1.091728  | 0.030926 | 0.855073  | 1.059135 | NaN       | NaN      | Na        |

### 3.3 데이터 재형성, 피벗

- 표 형식의 데이터를 재배포하는 연산( 재형성, 피벗 연산 )

- **stack** 메서드 : 데이터의 컬럼을 로우로 피벗(회전)
- **unstack** 메서드 : 로우를 컬럼으로 피벗(회전)

```
import numpy as np
import pandas as pd

mydf = pd.DataFrame([[80,90,70],[75,65,95]],
                    index = pd.Index(['kor','math'], name='subject'),
                    columns=pd.Index(['stu1','stu2','stu3'],name='student'))
print(mydf)
```

# 컬럼이 로우로 피벗되어 Series 객체 반환

```
stackdf = mydf.stack()
print(stackdf)
```

# 계층 색인의 안쪽에 있는 레벨부터 꺼내 컬럼으로 지정하여

# DataFrame 반환

```
unstackdf = stackdf.unstack()
print(unstackdf)
```

| student | stu1 | stu2 | stu3 |
|---------|------|------|------|
| subject |      |      |      |
| kor     | 80   | 90   | 70   |
| math    | 75   | 65   | 95   |

| subject | student |    |
|---------|---------|----|
| kor     | stu1    | 80 |
|         | stu2    | 90 |
|         | stu3    | 70 |
| math    | stu1    | 75 |
|         | stu2    | 65 |
|         | stu3    | 95 |

dtype: int64

| student | stu1 | stu2 | stu3 |
|---------|------|------|------|
| subject |      |      |      |
| kor     | 80   | 90   | 70   |
| math    | 75   | 65   | 95   |

## • 모든 값이 하위그룹에 없을 경우 누락데이터 발생

```
import numpy as np
import pandas as pd
```

```
mys1 = pd.Series([0,1,2,3], index=['num1','num2','num3','num4'])
mys2 = pd.Series([4,5,6], index=['num3','num4','num5'])
concat_sd = pd.concat([mys1,mys2], keys=['gp1','gp2'])
print(concat_sd)
```

# 해당 레벨에 있는 모든 값이 하위레벨에 속하지 않을 경우  
# unstack() 하면 누락 데이터 발생

```
unstack_sd = concat_sd.unstack()
print(unstack_sd)
```

# stack() : 누락된 데이터 자동으로 걸러 연산하기 때문에 원상 복구 용이  
stack\_sd = unstack\_sd.stack()  
print(stack\_sd)

# dropna=False 옵션으로 누락된 데이터 그대로 유지 할수도 있음  
stack\_sd = unstack\_sd.stack(dropna=False)  
print(stack\_sd)

|     |      |   |
|-----|------|---|
| gp1 | num1 | 0 |
|     | num2 | 1 |
|     | num3 | 2 |
|     | num4 | 3 |
| gp2 | num3 | 4 |
|     | num4 | 5 |
|     | num5 | 6 |

dtype: int64

|     | num1 | num2 | num3 | num4 | num5 |
|-----|------|------|------|------|------|
| gp1 | 0.0  | 1.0  | 2.0  | 3.0  | NaN  |
| gp2 | NaN  | NaN  | 4.0  | 5.0  | 6.0  |

|     |      |     |
|-----|------|-----|
| gp1 | num1 | 0.0 |
|     | num2 | 1.0 |
|     | num3 | 2.0 |
|     | num4 | 3.0 |
| gp2 | num3 | 4.0 |
|     | num4 | 5.0 |
|     | num5 | 6.0 |

dtype: float64

|     |      |     |
|-----|------|-----|
| gp1 | num1 | 0.0 |
|     | num2 | 1.0 |
|     | num3 | 2.0 |
|     | num4 | 3.0 |
|     | num5 | NaN |
| gp2 | num1 | NaN |
|     | num2 | NaN |
|     | num3 | 4.0 |
|     | num4 | 5.0 |
|     | num5 | 6.0 |

dtype: float64



### 3.4 pivot 메서드 : 데이터 재구조화(reshaping data)

```
import numpy as np
import pandas as pd
```

```
mydf = pd.read_excel('teacher_list_pivot_exam.xlsx')
print(mydf)
```

```
# index : 과정명, columns : 강사명 으로 재구조화
data_pivot = mydf.pivot(index='과정명',columns='강사명',
                           values='강의시수')
print(data_pivot)
```

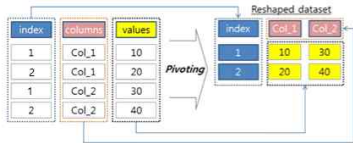
| 카테고리 | 과정명                          | 강의시수 | 강사명 |
|------|------------------------------|------|-----|
| 0    | prog C Programming           | 32   | 홍길동 |
| 1    | prog C Programming           | 32   | 대조영 |
| 2    | prog C Programming           | 28   | 이순신 |
| 3    | Emb Embedded C Programming   | 32   | 홍길동 |
| 4    | Emb Embedded C Programming   | 64   | 대조영 |
| 5    | Emb Embedded C Programming   | 36   | 이순신 |
| 6    | Emb Embedded Porting         | 28   | 홍길동 |
| 7    | Emb Embedded Porting         | 32   | 대조영 |
| 8    | Emb Embedded Porting         | 28   | 이순신 |
| 9    | Emb Linux system Programming | 70   | 홍길동 |
| 10   | Emb Linux system Programming | 25   | 대조영 |
| 11   | Emb Linux system Programming | 30   | 이순신 |

| 강사명                      | 대조영 | 이순신 | 홍길동 |
|--------------------------|-----|-----|-----|
| 과정명                      |     |     |     |
| C Programming            | 32  | 28  | 32  |
| Embedded C Programming   | 64  | 36  | 32  |
| Embedded Porting         | 32  | 28  | 28  |
| Linux system Programming | 25  | 30  | 70  |

데이터 재구조화 (Reshaping data by Pivoting)

`data.pivot(index, columns, values)`

`pd.pivot_table(data, index, columns, values)`



- pivot() 메서드 단점 : 중복된 색인(데이터)값이 있을 경우 ValueError 반환
- 단점을 보완한 집계연산(sum,mean등)이 가능한 pivot\_table() 메서드 사용 추천

**Thank You**

